

BIRZEIT UNIVERSITY

Faculty of Engineering and Technology

Department of Computer Science

COMP333—DATABASE SYSTEMS

Phase 2: Preliminary Design Document

Prepared by:

Yahya Hasan 1242481 Section 5

Mohammed Quraan 1243436 Section 5

Instructor: Yousef Hassouneh

Date: April 28, 2026

Contents

1	Project Description	2
2	Entity Sets	3
3	E-R Diagram	5
4	Relational Schemas	7
5	Constraints	8
5.1	Primary-Key Constraints	8
5.2	Foreign-Key Constraints	9
5.3	Additional Domain and Integrity Constraints	10

1 Project Description

Bawabet Al-Quds Roastery is a specialty coffee roastery and distribution business based in Ramallah, Palestine, offering coffee, roasted goods, spices, and curated packages to both individual retail customers and wholesale/B2B clients such as supermarkets and distributors.

The system supports two user roles: a **customer**, who browses the catalog, manages a shopping cart, places orders, and leaves product reviews; and an **admin**, who controls the catalog, creates promotional packages, manages branches and staff, and has a global view of all platform activity. Customers are specialised into **Individual** (retail) and **Business** (wholesale) subtypes, with business clients storing a registered business address via the shared *Location* entity.

The roastery operates through multiple **branches**, each with its own staff, inventory, and purchase orders. Branches issue **purchase orders** to **suppliers** and run **roast batches** that produce the *Coffee* and *Roasts* catalog items. Inter-branch inventory movement is handled via **stock transfers**. The product catalog is modelled with a single **Item** superclass specialised into *Coffee*, *Roasts*, *Spice*, and *Package*.

Orders capture a snapshot of cart items at checkout and are linked to a **delivery** record whose destination is drawn from the shared *Location* entity. Customers may save **payment** card details, and the system tracks product **reviews** for customer feedback. **Discount** codes can be applied to orders.

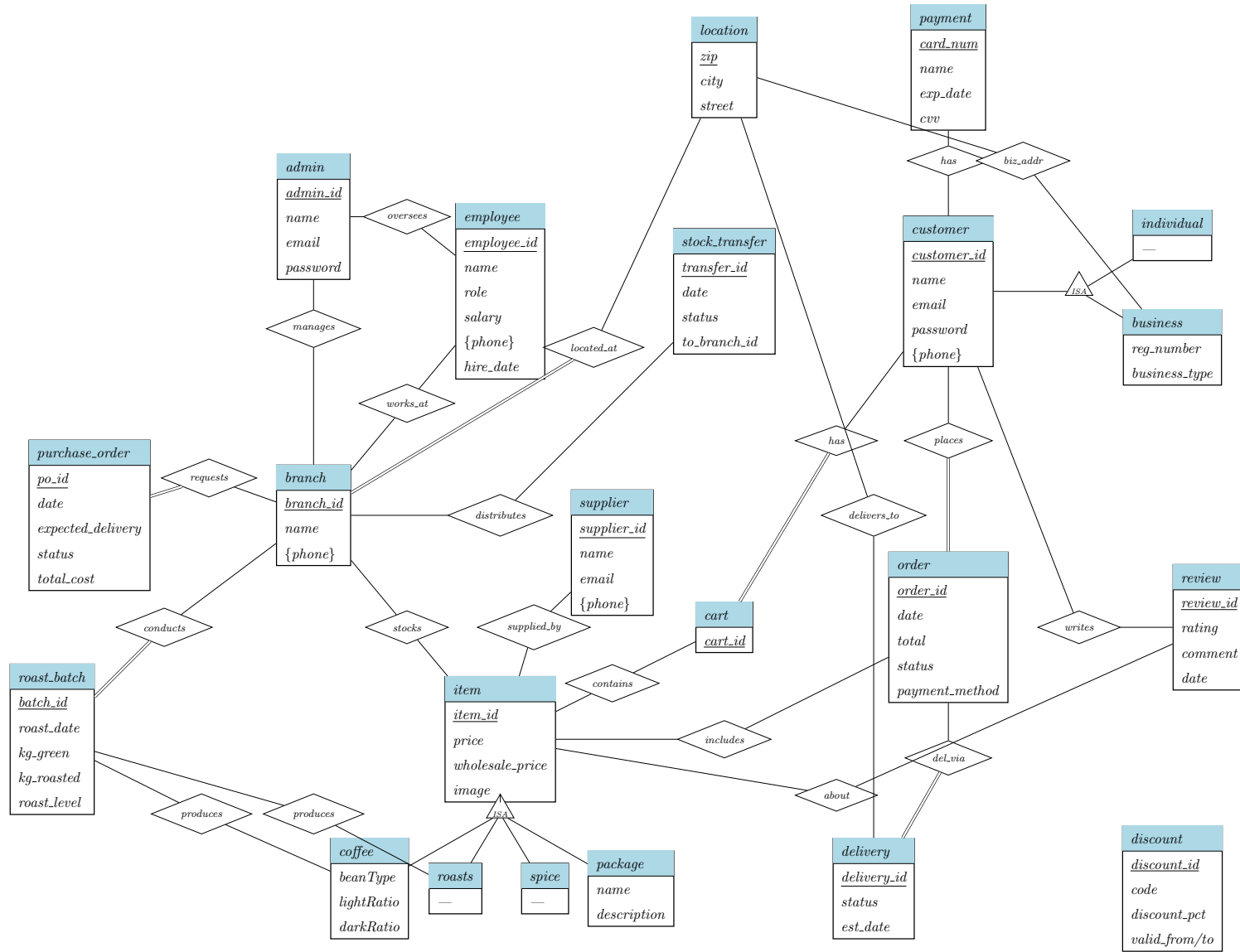
2 Entity Sets

Entity	Description
Item	Superclass for every sellable unit. Holds <i>item_id</i> , <i>price</i> , <i>whole-sale_price</i> , and <i>image</i> . Specialised into <i>Coffee</i> , <i>Roasts</i> , <i>Spice</i> , and <i>Package</i> via an ISA hierarchy.
Coffee	Subtype of Item for roasted coffee products. Adds <i>beanType</i> , <i>lightRatio</i> , and <i>darkRatio</i> . Linked to the <i>RoastBatch</i> that produced it.
Roasts	Subtype of Item for other roasted goods (e.g. nuts, grains). No additional attributes. Linked to the producing <i>RoastBatch</i> .
Spice	Subtype of Item for spice and herb products. No additional attributes beyond Item.
Package	Subtype of Item for admin-assembled bundles. Adds <i>name</i> and <i>description</i> ; sold as a single catalog unit.
Customer	Superclass for all registered buyers. Holds <i>customer_id</i> , <i>name</i> , <i>email</i> , <i>password</i> , and multivalued <i>phone</i> . Specialised into <i>Individual</i> and <i>Business</i> .
Individual	Subtype of Customer for retail buyers. No additional attributes.
Business	Subtype of Customer for wholesale/B2B clients. Adds <i>reg_number</i> and <i>business_type</i> . Has a registered business address linked via <i>Location</i> .
Admin	Administrative account. Holds <i>admin_id</i> , <i>name</i> , <i>email</i> , and <i>password</i> . Manages branches and oversees employees.
Branch	A physical roastery location. Holds <i>branch_id</i> , <i>name</i> , and multivalued <i>phone</i> . Must have exactly one <i>Location</i> (total participation in <i>located_at</i>).
Employee	Staff member assigned to a branch. Holds <i>employee_id</i> , <i>name</i> , <i>role</i> , <i>salary</i> , multivalued <i>phone</i> , and <i>hire_date</i> .
Supplier	External vendor supplying items to the roastery. Holds <i>supplier_id</i> , <i>name</i> , <i>email</i> , and multivalued <i>phone</i> .
PurchaseOrder	Procurement order issued by a branch. Holds <i>po_id</i> , <i>date</i> , <i>expected_delivery</i> , <i>status</i> , and <i>total_cost</i> . Total participation with Branch (every PO must come from a branch).
RoastBatch	A production run at a branch. Holds <i>batch_id</i> , <i>roast_date</i> , <i>kg_green</i> , <i>kg_roasted</i> , and <i>roast_level</i> . Total participation with Branch.
Cart	Active shopping basket. Holds <i>cart_id</i> . Total participation with Customer (a cart must belong to a customer).

Entity	Description
Order	Confirmed purchase. Holds <i>order_id</i> , <i>date</i> , <i>total</i> , <i>status</i> (Pending/Fulfilled/Cancelled), and <i>payment_method</i> . Total participation with both Customer and Delivery.
Delivery	Shipping record for an order. Holds <i>delivery_id</i> , <i>status</i> , and <i>est_date</i> . Destination address is drawn from <i>Location</i> via <i>delivers_to</i> .
StockTransfer	Inter-branch inventory movement. Holds <i>transfer_id</i> , <i>date</i> , <i>status</i> , and <i>to_branch_id</i> . The source branch is captured by the <i>distributes</i> relationship.
Payment	Saved payment card belonging to a customer. Identified by <i>card_num</i> ; holds <i>name</i> , <i>exp_date</i> , and <i>cvv</i> .
Location	Reusable geographic address. Identified by <i>zip</i> ; holds <i>city</i> and <i>street</i> . Shared by Branch (<i>located_at</i>), Business (<i>biz_addr</i>), and Delivery (<i>delivers_to</i>).
Review	Customer review of a catalog item. Holds <i>review_id</i> , <i>rating</i> , <i>comment</i> , and <i>date</i> . Linked to both the writing customer and the reviewed item.
Discount	Promotional discount code. Holds <i>discount_id</i> , <i>code</i> , <i>discount_pct</i> , and a validity period (<i>valid_from</i> , <i>valid_to</i>).

Table 1: Entity sets and their descriptions.

3 E-R Diagram



4 Relational Schemas

Each schema is written as *Relation*(primary key, attributes). Foreign-key attributes are shown in *italics*. Multivalued attributes (e.g. phone numbers) would each be stored in a separate relation in a fully normalised implementation; they are noted here as {*attr*}.

- R1. Location**(zip, city, street)
- R2. Admin**(admin_id, name, email, password)
- R3. Supplier**(supplier_id, name, email, {phone})
- R4. Branch**(branch_id, *admin_id*, *zip*, name, {phone})
- R5. Employee**(employee_id, *branch_id*, name, role, salary, {phone}, hire_date)
- R6. PurchaseOrder**(po_id, *branch_id*, date, expected_delivery, status, total_cost)
- R7. RoastBatch**(batch_id, *branch_id*, roast_date, kg_green, kg_roasted, roast_level)
- R8. SuppliedBy**(*supplier_id*, *item_id*)
(*M:N — resolves the supplied_by relationship*)
- R9. Item**(item_id, price, wholesale_price, image, item_type)
item_type ∈ {coffee, roasts, spice, package}
- R10. Coffee**(*item_id*, *batch_id*, beanType, lightRatio, darkRatio)
- R11. Roasts**(*item_id*, *batch_id*)
- R12. Spice**(*item_id*)
- R13. Package**(*item_id*, name, description)
- R14. BranchStock**(*branch_id*, *item_id*, qty)
(*M:N with qty attribute — resolves the stocks relationship*)
- R15. Customer**(customer_id, name, email, password, {phone})
- R16. Individual**(*customer_id*)
- R17. Business**(*customer_id*, *zip*, reg_number, business_type)
- R18. Payment**(card_num, *customer_id*, name, exp_date, cvv)
- R19. Cart**(cart_id, *customer_id*)
- R20. CartItem**(*cart_id*, *item_id*, qty)
(*M:N — resolves the contains relationship*)
- R21. Order**(order_id, *customer_id*, date, total, status, payment_method)
- R22. OrderItem**(*order_id*, *item_id*, qty, unit_price)
(*M:N with qty and price snapshot — resolves the includes relationship*)
- R23. Delivery**(delivery_id, *order_id*, *zip*, status, est_date)
- R24. StockTransfer**(transfer_id, *from_branch_id*, *to_branch_id*, date, status)
- R25. Review**(review_id, *customer_id*, *item_id*, rating, comment, date)

R26. Discount(discount_id, code, discount_pct, valid_from, valid_to)

5 Constraints

5.1 Primary-Key Constraints

Every relation above has a single-attribute primary key except the five intersection tables (**SuppliedBy**, **BranchStock**, **CartItem**, **OrderItem**) and the ISA relations (**Individual**, **Business**, **Coffee**, **Roasts**, **Spice**, **Package**), which inherit the parent's primary key.

5.2 Foreign-Key Constraints

Relation	Foreign Key	References
Branch	admin_id	Admin(admin_id)
Branch	zip	Location(zip)
Employee	branch_id	Branch(branch_id)
PurchaseOrder	branch_id	Branch(branch_id)
RoastBatch	branch_id	Branch(branch_id)
SuppliedBy	supplier_id	Supplier(supplier_id)
SuppliedBy	item_id	Item(item_id)
BranchStock	branch_id	Branch(branch_id)
BranchStock	item_id	Item(item_id)
Coffee	item_id	Item(item_id)
Coffee	batch_id	RoastBatch(batch_id)
Roasts	item_id	Item(item_id)
Roasts	batch_id	RoastBatch(batch_id)
Spice	item_id	Item(item_id)
Package	item_id	Item(item_id)
Individual	customer_id	Customer(customer_id)
Business	customer_id	Customer(customer_id)
Business	zip	Location(zip)
Payment	customer_id	Customer(customer_id)
Cart	customer_id	Customer(customer_id)
CartItem	cart_id	Cart(cart_id)
CartItem	item_id	Item(item_id)
Order	customer_id	Customer(customer_id)
OrderItem	order_id	Order(order_id)
OrderItem	item_id	Item(item_id)
Delivery	order_id	Order(order_id)
Delivery	zip	Location(zip)
StockTransfer	from_branch_id	Branch(branch_id)
StockTransfer	to_branch_id	Branch(branch_id)
Review	customer_id	Customer(customer_id)
Review	item_id	Item(item_id)

Table 2: Foreign-key constraints.

5.3 Additional Domain and Integrity Constraints

- `Item.item_type` $\in \{\text{'coffee'}, \text{'roasts'}, \text{'spice'}, \text{'package'}\}$
- `Order.status` $\in \{\text{'Pending'}, \text{'Fulfilled'}, \text{'Cancelled'}\}$
- `PurchaseOrder.status` $\in \{\text{'Pending'}, \text{'Received'}, \text{'Cancelled'}\}$
- `Delivery.status` $\in \{\text{'Processing'}, \text{'Shipped'}, \text{'Delivered'}\}$
- `StockTransfer.status` $\in \{\text{'In Transit'}, \text{'Completed'}, \text{'Cancelled'}\}$
- `Item.price` > 0 ; `Item.wholesale_price` > 0
- `BranchStock.qty` ≥ 0
- `CartItem.qty` > 0
- `OrderItem.qty` > 0 ; `OrderItem.unit_price` > 0
- `RoastBatch.kg_green` > 0 ; `RoastBatch.kg_roasted` > 0
- `RoastBatch.kg_roasted` $\leq$ `RoastBatch.kg_green`
- `Discount.discount_pct` $\in (0, 100]$; `valid_from` $\leq$ `valid_to`
- `Review.rating` $\in \{1, 2, 3, 4, 5\}$
- `Cart.customer_id` is UNIQUE — each customer has at most one active cart at a time.
- `Delivery.order_id` is UNIQUE — one delivery per order (total participation enforced by NOT NULL).
- On order confirmation: `BranchStock.qty` must be decremented by the ordered quantity. A check constraint ensures `qty` ≥ 0 at all times (no overselling).